

UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e de Tecnologia

ARQUITETURAS DE ALTO DESEMPENHO

Relatório das Práticas 1 e 2: Cálculo de Pi por Monte Carlo e por Leibniz

Docente: Prof. Dr. Emerson Carlos Pedrino

Discentes

Enzo Dezem Alves

RA: 801743

Curso: Engenharia de Computação

Gustavo de Oliveria Gimenes

RA: 820759

Curso: Engenharia de Computação

São Carlos, 25 de maio de 2026

1 Introdução

A crescente demanda por desempenho computacional em áreas como simulação científica, análise de risco, processamento de sinais e modelagem numérica tem impulsionado a adoção de estratégias paralelas e vetoriais nas arquiteturas modernas. Em vez de depender exclusivamente do aumento de frequência de clock, os sistemas contemporâneos passaram a explorar paralelismo em diferentes níveis, como paralelismo de dados, paralelismo de instruções e execução em múltiplos núcleos.

Nesta prática, foram implementados e comparados dois métodos para estimar o valor de π : o método probabilístico de Monte Carlo e o método determinístico da Série de Leibniz. Para avaliar o impacto das escolhas de implementação e dos recursos arquiteturais, as soluções foram desenvolvidas utilizando diferentes abordagens: versões seriais (baseadas em laços de repetição), versões vetoriais em Octave e versões utilizando bibliotecas otimizadas como o NumPy em Python, além de processamento paralelo. O objetivo é observar, na prática, como essas estratégias afetam o tempo de execução, a aceleração (*speedup*), a eficiência e o erro numérico.

2 Desenvolvimento Teórico

2.1 Ideia do método de Monte Carlo

O método de Monte Carlo é uma técnica estatística baseada em amostragem aleatória. Em vez de calcular uma grandeza de forma determinística e analítica, a ideia é estimá-la a partir da frequência com que eventos aleatórios satisfazem uma condição geométrica ou probabilística.

Para estimar π , considera-se um quadrado de lado 2 com um quarto de círculo de raio 1 inscrito no primeiro quadrante. A área do quadrado é 4, enquanto a área do quarto de círculo é $\pi/4$. Assim, a razão entre as áreas é:

$$\frac{A_q}{A_s} = \frac{\pi/4}{1} = \frac{\pi}{4} \quad (1)$$

Se lançarmos N pontos aleatórios uniformemente distribuídos no quadrado, a fração de pontos que caem dentro do quarto de círculo aproxima a razão entre as áreas. Se N_{in} representa a quantidade de pontos com $x^2 + y^2 \leq 1$, então:

$$\pi \approx 4 \cdot \frac{N_{in}}{N} \quad (2)$$

A figura apresentada abaixo ilustra exatamente essa interpretação geométrica: os pontos gerados no quadrado são classificados como internos ou externos ao arco do quarto de círculo, e essa contagem fornece a estimativa de π .

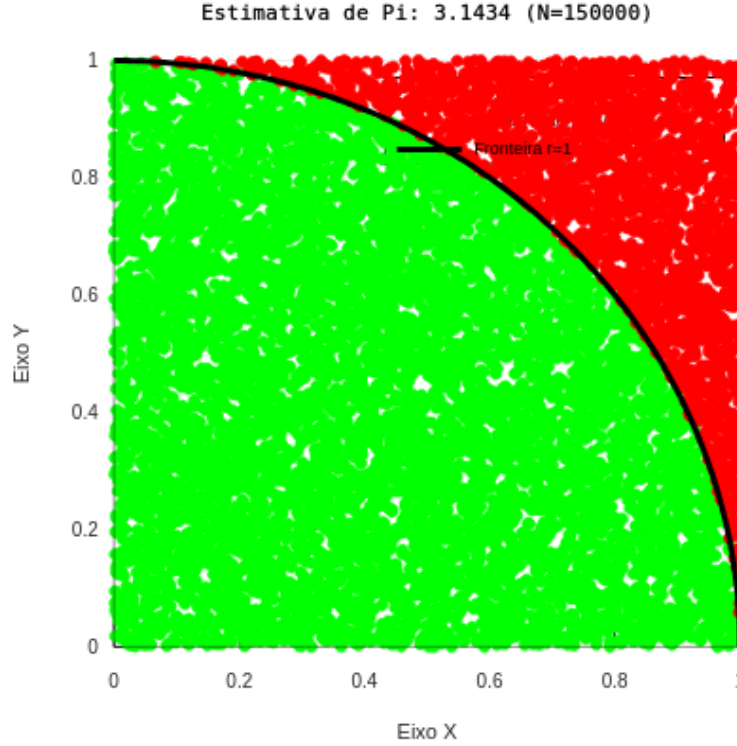


Figura 1: Distribuição dos pontos aleatórios no quadrado unitário. Os pontos dentro do quarto de círculo são utilizados para estimar o valor de π .

2.2 Ideia do método da Série de Leibniz

Diferentemente de Monte Carlo, que utiliza amostragem probabilística, o método de Leibniz para o cálculo de π é uma abordagem puramente determinística. Ele baseia-se na avaliação dos termos de uma série infinita alternada.

A dedução da fórmula parte da expansão em série de Taylor para a função arco-tangente. Sabemos que a derivada geométrica dessa função é dada por:

$$\frac{d}{dx} \arctan(x) = \frac{1}{1+x^2} \quad (3)$$

Para o intervalo onde $|x| < 1$ (e no limite de convergência $x = 1$), essa fração pode ser representada como a soma de uma série geométrica infinita de razão $-x^2$:

$$\frac{1}{1+x^2} = \sum_{k=0}^{\infty} (-1)^k x^{2k} \quad (4)$$

Integrando ambos os lados da equação de 0 a x , obtemos a série de Taylor para a função arco-tangente:

$$\arctan(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{2k+1} \quad (5)$$

Sabendo que $\arctan(1) = \frac{\pi}{4}$, basta substituir $x = 1$ na equação acima para encontrar a clássica Série de Leibniz:

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \quad (6)$$

Multiplicando a equação por 4, isolamos π , obtendo a fórmula final que será implementada computacionalmente:

$$\pi = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} \quad (7)$$

2.3 Comparação entre Monte Carlo e Leibniz

Embora ambos os métodos permitam estimar π , eles possuem naturezas computacionais muito distintas. O método de Monte Carlo é estocástico, dependendo pesadamente da geração de números pseudoaleatórios. Isso embute um custo computacional elevado por iteração e gera variabilidade nos resultados, com o erro diminuindo a uma taxa proporcional a $1/\sqrt{N}$.

Por outro lado, a Série de Leibniz é determinística e produzirá exatamente o mesmo resultado para um mesmo número de termos n . Contudo, a série apresenta uma convergência extremamente lenta, exigindo milhões de iterações para estabilizar as primeiras casas decimais de π .

2.4 Erro da aproximação

A análise de erro difere fundamentalmente dependendo da natureza do método. No caso do método de Monte Carlo, que é probabilístico, o resultado não é exato e a qualidade da estimativa cresce com o número de pontos (N), pois a lei dos grandes números reduz a variabilidade amostral. Já no método de Leibniz, que é determinístico, o erro decorre do truncamento de uma série infinita; a precisão aumenta conforme o número de termos (n) calculados cresce. Em ambos os casos, quanto maior o esforço computacional, menor tende a ser o erro absoluto entre o valor estimado e o valor real.

O erro absoluto pode ser calculado por:

$$E_{abs} = |\pi - \pi_{est}| \quad (8)$$

Já o erro relativo percentual é dado por:

$$E_{rel}(\%) = \frac{|\pi - \pi_{est}|}{\pi} \cdot 100 \quad (9)$$

Na prática analisada, para um número elevado de iterações (N ou n), os valores

estimados ficam muito próximos de π . No entanto, o método de Monte Carlo apresenta pequenas variações a cada execução devido à aleatoriedade do processo, enquanto Leibniz entrega sempre o mesmo resultado para um mesmo n , convergindo de forma monotônica, porém lenta.

2.5 Vetorização e processamento SIMD

A vetorização consiste em operar sobre blocos de dados ao mesmo tempo, ao invés de processar cada elemento individualmente em laços interpretados. Em arquiteturas modernas, isso se relaciona ao conceito de SIMD (*Single Instruction, Multiple Data*), no qual uma mesma instrução é aplicada simultaneamente a vários dados.

No contexto desta prática, as versões vetoriais em Octave substituem os laços explícitos por operações sobre vetores completos. Para Monte Carlo, isso significa calcular as coordenadas quadráticas ($x^2 + y^2$) de milhares de pontos de uma só vez. Para o método de Leibniz, significa calcular múltiplos termos da série $((-1)^k / (2k + 1))$ simultaneamente a partir de um vetor de índices. Isso reduz drasticamente o custo de interpretação de instruções e explora melhor as unidades lógicas e aritméticas (ALUs) modernas, tornando as versões vetoriais muito mais rápidas que as seriais puras.

2.6 Paralelismo, multiprocessamento e MIMD

Quando o problema é dividido em partes independentes e cada parte é processada por um núcleo diferente, temos um caso típico de paralelismo de dados. Essa abordagem se encaixa em MIMD (*Multiple Instruction, Multiple Data*), pois múltiplos fluxos de instruções executam sobre múltiplos conjuntos de dados.

Na implementação com Python, a biblioteca *multiprocessing* foi utilizada para permitir execução real em múltiplos processos. Essa estratégia é altamente eficaz para ambos os métodos avaliados: no Monte Carlo, a geração e contagem de pontos é dividida entre os núcleos; no método de Leibniz, o somatório é particionado em subintervalos de k , com cada núcleo calculando uma soma parcial que, ao final, é agregada.

2.7 Lei de Amdahl

A Lei de Amdahl descreve o limite teórico de aceleração ao paralelizar um programa. Se uma fração f do código for serial e a fração restante for paralelizável, o speedup máximo com p processadores é:

$$S = \frac{1}{f + \frac{1-f}{p}} \quad (10)$$

A Lei de Amdahl demonstra que, mesmo quando a parte paralela cresce muito, a parcela serial impõe um teto ao ganho total. Isso explica por que a aceleração real nem sempre cresce de forma perfeitamente linear com o número de núcleos.

Tanto na simulação de Monte Carlo quanto na expansão de Leibniz, o núcleo matemático é quase inteiramente paralelizável (altamente escalável). Contudo, persistem custos não paralelizáveis, como a inicialização do ambiente Python, a importação de bibliotecas, a alocação de memória para os processos, a divisão do trabalho e a redução (coleta e soma) dos resultados finais. Esses custos compõem o *overhead* do paralelismo e explicam por que a eficiência observada ($E = S/p$) costuma ficar abaixo de 100%.

2.8 Relação com a arquitetura estudada

O tema da prática se encaixa perfeitamente no estudo de arquiteturas de alto desempenho porque combina a avaliação de diferentes abordagens algorítmicas com a exploração de recursos de hardware. Ao contrastar as implementações seriais, vetoriais e multiprocessadas de dois métodos matemáticos distintos, evidencia-se que o desempenho atual depende menos do aumento isolado da frequência de *clock* e muito mais da adoção de técnicas arquiteturais, como a exploração eficiente de instruções SIMD e a orquestração de sistemas MIMD em processadores de múltiplos núcleos.

3 Desenvolvimento Prático

Para o método de Monte Carlo, foram utilizados N pontos aleatórios a depender do ambiente utilizado. O procedimento seguiu os passos abaixo:

1. geração dos pontos (x, y) uniformemente distribuídos em $[0, 1] \times [0, 1]$;
2. verificação da condição $x^2 + y^2 \leq 1$;
3. contagem dos pontos internos ao quarto de círculo;
4. estimativa de π pela razão entre pontos internos e pontos totais;
5. medição do tempo de execução para cada abordagem.

Para o método da Série de Leibniz, o experimento avaliou um número n de termos variável. O procedimento consistiu em:

1. geração de um vetor de índices k variando de 0 até $n - 1$;
2. cálculo do termo da série para cada índice iterativamente (serial) ou matricialmente (vetorial/SIMD/Multi-core);

3. somatório dos termos calculados e multiplicação por 4 para estimar π ;
4. medição do tempo de execução das abordagens.

O roteiro da prática orienta o uso de vetores no Octave e a medição com `tic-toc`; também solicita a comparação com a implementação serial e a repetição em Python utilizando bibliotecas otimizadas como `numpy` e a biblioteca `multiprocessing`.

3.1 Implementação em Octave

3.1.1 Método de Monte Carlo

A execução do algoritmo de Monte Carlo em Octave foi projetada utilizando três abordagens distintas: serial e vetorial. O experimento utilizou 150.000 pontos aleatórios para a estimativa de π .

Devido às restrições de ambiente e limitações de alocação de hardware (contêineres de núcleo único) da plataforma Octave Online, não foi possível instanciar múltiplas cores. Portanto, a abordagem multi-core não pôde ser executada.

Dessa forma, a saída do programa apresentou os seguintes resultados práticos:

```
--- Abordagem Serial ---
Tempo: 3.0742 segundos (Pi = 3.141040)

--- Abordagem Vetorial ---
Tempo: 0.0066 segundos (Pi = 3.148053)

--- Resultados da Análise de Desempenho ---
Erro Absoluto (Serial): 0.000553
Erro Absoluto (Vetorial): 0.006461
Speedup Vetorial vs Serial: 466.79x mais rápido
```

Observa-se uma diferença drástica de desempenho entre as abordagens avaliadas. A versão serial apresentou o maior tempo de execução, justificado pelo elevado *overhead* do interpretador ao processar o cálculo de cada ponto individualmente em um laço iterativo (*for loop*). Em contrapartida, a versão vetorial obteve uma redução expressiva no tempo de execução, tirando proveito das operações de matrizes e vetores otimizadas internamente em linguagens de baixo nível pelo núcleo do Octave.

O código-fonte utilizado para esta simulação é apresentado a seguir:

```
1 n = 150000;
2 r = 1;
3 fprintf('Simulando com n = %d pontos...\n\n', n);
```

```

4
5 % SERIAL
6 N_dentro_serial = 0;
7 for i = 1:n
8     x = rand();
9     y = rand();
10    if (x^2 + y^2 <= r^2)
11        N_dentro_serial = N_dentro_serial + 1;
12    end
13 end
14 pi_serial = 4 * (N_dentro_serial / n);
15 tempo_serial = toc;
16
17 fprintf('--- Abordagem Serial ---\n');
18 fprintf('Tempo: %.4f segundos (Pi = %.6f)\n\n', tempo_serial, pi_serial
    );
19
20 % VETORIAL
21 tic;
22 x_vet = rand(n, 1);
23 y_vet = rand(n, 1);
24
25 % Calcula e guarda quais pontos caíram dentro
26 dentro_idx = (x_vet.^2 + y_vet.^2) <= r^2;
27 N_dentro_vetorial = sum(dentro_idx);
28
29 pi_vetorial = 4 * (N_dentro_vetorial / n);
30 tempo_vetorial = toc;
31
32 fprintf('--- Abordagem Vetorial ---\n');
33 fprintf('Tempo: %.4f segundos (Pi = %.6f)\n\n', tempo_vetorial,
    pi_vetorial);
34
35 % ANALISE COMPLETA: SPEEDUP E ERRO
36 fprintf('--- Resultados da Analise de Desempenho ---\n');
37
38 % Erros
39 fprintf('Erro Absoluto (Serial): %.6f\n', abs(pi - pi_serial));
40 fprintf('Erro Absoluto (Vetorial): %.6f\n', abs(pi - pi_vetorial));
41
42 % Speedups
43 speedup_vetorial = tempo_serial / tempo_vetorial;
44
45 fprintf('Speedup Vetorial vs Serial: %.2fx mais rapido\n',
    speedup_vetorial);
46
47 % PLOTAGEM

```

```

48 figure('Name', 'Monte Carlo - Primeiro Quadrante');
49 hold on;
50
51 % Plota uma amostra de 10.000 pontos para nao sobrecarregar o grafico
52 amostra = min(10000, n);
53 idx_dentro = dentro_idx(1:amostra);
54
55 % Usando os vetores x_vet e y_vet gerados na etapa 2
56 plot(x_vet(1:amostra)(idx_dentro), y_vet(1:amostra)(idx_dentro), 'g.',
      'MarkerSize', 6);
57 plot(x_vet(1:amostra)(~idx_dentro), y_vet(1:amostra)(~idx_dentro), 'r
      .', 'MarkerSize', 6);
58
59 % Desenha o arco do circulo (1/4 de circulo)
60 angulo = 0:pi/200:pi/2;
61 plot(r*cos(angulo), r*sin(angulo), 'k', 'LineWidth', 3);
62
63 xlabel('Eixo X');
64 ylabel('Eixo Y');
65 axis([0 1 0 1], 'square');
66 grid on;
67 title(['Estimativa de Pi: ', num2str(pi_vetorial), ' (N=', num2str(n),
      ')']);
68 legend('Pontos Internos (In)', 'Pontos Externos (Out)', 'Fronteira r
      =1');

```

3.1.2 Método de Leibniz

Para o método de Leibniz, a execução no Octave avaliou as duas abordagens variando a carga computacional (N termos). Os resultados obtidos foram:

```

=====
SIMULACAO PARA N = 100000
=====
Pi estimado (Vetorial): 3.14158265
Erro da aproximacao    : 0.00001000
Tempo Sequencial       : 2.4286 segundos
Tempo Vetorial         : 0.0690 segundos
Speedup                : 35.20x mais rapido

=====
SIMULACAO PARA N = 150000
=====

```

```

Pi estimado (Vetorial): 3.14158599
Erro da aproximacao   : 0.00000667
Tempo Sequencial      : 3.3942 segundos
Tempo Vetorial        : 0.0572 segundos
Speedup               : 59.32x mais rapido

```

```

=====
SIMULACAO PARA N = 200000
=====

```

```

Pi estimado (Vetorial): 3.14158765
Erro da aproximacao   : 0.00000500
Tempo Sequencial      : 3.4984 segundos
Tempo Vetorial        : 0.1008 segundos
Speedup               : 34.71x mais rapido

```

A análise dos resultados evidencia a imensa superioridade da abordagem vetorial em relação à sequencial no ambiente do Octave. Em todos os cenários testados, o ganho de desempenho (*speedup*) foi expressivo, mantendo-se consistentemente acima de 34x. A oscilação do *speedup* (atingindo um pico de 59.32x para 150.000 termos) é um comportamento comum quando lidamos com tempos de execução vetorial na casa dos milissegundos, tornando as medições sensíveis a pequenas flutuações no uso da CPU ou concorrência de processos no sistema operacional subjacente.

Do ponto de vista matemático, o experimento ilustra perfeitamente a lenta convergência do método de Leibniz. Conforme dobramos a carga computacional de 100.000 para 200.000 termos, o erro absoluto da aproximação cai de maneira linear e inversamente proporcional (de 0.00001000 para 0.00000500), alcançando o valor mais preciso de π neste teste: 3.14158765.

O código-fonte utilizado para esta simulação é apresentado a seguir:

```

1 % Vetor com os valores de N que serao testados
2 valores_N = [100000, 150000, 200000];
3
4 fprintf('Iniciando simulacoes para variacoes de N...\n\n');
5
6 % Laco para percorrer cada valor de N no vetor
7 for k = 1:length(valores_N)
8
9     N = valores_N(k);
10
11     fprintf('=====\n');
12     fprintf(' SIMULACAO PARA N = %d \n', N);
13     fprintf('=====\n');

```

```

14
15 % SEQUENCIAL
16 tic;
17 soma_seq = 0;
18 for i = 0:(N-1)
19     termo = ((-1)^i) / (2*i + 1);
20     soma_seq = soma_seq + termo;
21 end
22 pi_seq = 4 * soma_seq;
23 tempo_seq = toc;
24
25 % VETORIAL
26 tic;
27 vetor_i = 0:(N-1);
28 termos_vet = ((-1).^vetor_i) ./ (2.*vetor_i + 1);
29 pi_vetorial = 4 * sum(termos_vet);
30 tempo_vet = toc;
31
32 % ANALISE DE DESEMPENHO E ERRO
33 erro_absoluto = abs(pi - pi_vetorial);
34
35 % Protecao contra divisao por zero caso tempo_vet seja 0.0000
36 if tempo_vet > 0
37     speedup = tempo_seq / tempo_vet;
38 else
39     speedup = 0;
40 end
41
42 % Exibicao dos resultados para este N especifico
43 fprintf('Pi estimado (Vetorial): %.8f\n', pi_vetorial);
44 fprintf('Erro da aproximacao : %.8f\n', erro_absoluto);
45 fprintf('Tempo Sequencial : %.4f segundos\n', tempo_seq);
46 fprintf('Tempo Vetorial : %.4f segundos\n', tempo_vet);
47
48 if speedup > 0
49     fprintf('Speedup : %.2fx mais rapido\n\n',
50             speedup);
51 else
52     fprintf('Speedup : N/A (tempo vetorial muito
53             proximo de zero)\n\n');
54 end
end

```

3.2 Implementação em Python

Nesta etapa, avaliou-se o desempenho dos métodos de Monte Carlo e Leibniz sob três paradigmas de programação em Python: serial (utilizando laço `for` nativo), vetorial (utilizando a biblioteca otimizada `numpy`) e paralela (utilizando a biblioteca `multiprocessing` para distribuir a carga entre 8 núcleos de processamento).

3.2.1 Método de Monte Carlo

Os resultados obtidos para a simulação de Monte Carlo com 10.000.000 de pontos foram:

```
--- Abordagem Serial ---
Tempo: 2.4393 segundos (Pi = 3.142000)
Erro da aproximacao: 0.00040735

--- Abordagem Vetorial (Numpy) ---
Tempo: 0.3973 segundos (Pi = 3.142055)
Erro da aproximacao: 0.00046255

--- Abordagem Multi-Core (Multiprocessing) ---
Tempo: 0.3153 segundos (Pi = 3.141544)
Erro da aproximacao: 0.00004905
Nucleos utilizados: 8

--- Resultados Finais ---
Speedup Vetorial vs Serial : 6.14x mais rapido
Speedup Multi vs Serial    : 7.74x mais rapido
Speedup Multi vs Vetorial  : 1.26x mais rapido
```

A análise dos tempos demonstra que a abordagem vetorial apresentou um ganho expressivo (5.85x mais rápida que a serial), evidenciando a lentidão característica do interpretador Python para laços de repetição puros. A versão multi-core obteve o melhor tempo absoluto (0.3577 s), resultando no maior *speedup* geral (6.48x). Contudo, o ganho da versão multi-core sobre a puramente vetorial foi modesto (apenas 1.11x), indicando que o custo (*overhead*) de criar e sincronizar 8 processos paralelos consumiu boa parte do benefício computacional para este volume de dados.

O código-fonte utilizado para esta simulação é apresentado a seguir:

```
1 import time
2 import random
3 import multiprocessing as mp
4 import numpy as np
```

```

5 import math
6
7 def worker_monte_carlo(n_pontos):
8     np.random.seed()
9     x = np.random.rand(n_pontos)
10    y = np.random.rand(n_pontos)
11    return np.sum(x**2 + y**2 <= 1.0)
12
13 if __name__ == '__main__':
14
15     N = 100000000
16     print(f"Iniciando simulacao Monte Carlo com N = {N} pontos...\n")
17
18     # SERIAL
19     inicio = time.time()
20     dentro_serial = 0
21     for _ in range(N):
22         x = random.random()
23         y = random.random()
24         if x*x + y*y <= 1.0:
25             dentro_serial += 1
26
27     pi_serial = 4 * (dentro_serial / N)
28     tempo_serial = time.time() - inicio
29
30     print("--- Abordagem Serial ---")
31     print(f"Tempo: {tempo_serial:.4f} segundos (Pi = {pi_serial:.6f})\n")
32
33     # VETORIAL
34     inicio = time.time()
35
36     # Numpy gera os arrays e faz operacoes vetorizadas em C por baixo
37     dos panos
38     x_vet = np.random.rand(N)
39     y_vet = np.random.rand(N)
40     dentro_vet = np.sum(x_vet**2 + y_vet**2 <= 1.0)
41
42     pi_vet = 4 * (dentro_vet / N)
43     tempo_vet = time.time() - inicio
44
45     print("--- Abordagem Vetorial (Numpy) ---")
46     print(f"Tempo: {tempo_vet:.4f} segundos (Pi = {pi_vet:.6f})\n")
47
48     # MULTI-CORE
49     inicio = time.time()

```

```

50     # Descobre quantos nucleos de CPU o seu computador tem
51     num_cores = mp.cpu_count()
52
53     # Divide o total de pontos igualitariamente entre os nucleos
54     pontos_por_core = [N // num_cores] * num_cores
55     # Adiciona o resto da divisao no primeiro nucleo (se houver)
56     pontos_por_core[0] += N % num_cores
57
58     # Cria o Pool de processos e mapeia a funcao worker para a lista de
        divisoes
59     with mp.Pool(processes=num_cores) as pool:
60         resultados = pool.map(worker_monte_carlo, pontos_por_core)
61
62     # Soma os resultados de todos os processos
63     dentro_multi = sum(resultados)
64
65     pi_multi = 4 * (dentro_multi / N)
66     tempo_multi = time.time() - inicio
67
68     print("--- Abordagem Multi-Core (Multiprocessing) ---")
69     print(f"Tempo: {tempo_multi:.4f} segundos (Pi = {pi_multi:.6f})")
70     print(f"Nucleos utilizados: {num_cores}\n")
71
72     # ANALISE DE DESEMPENHO E SPEEDUP
73     print("--- Resultados Finais ---")
74     print(f"Speedup Vetorial vs Serial    : {(tempo_serial/tempo_vet):.2f}
        }x mais rapido")
75     print(f"Speedup Multi vs Serial      : {(tempo_serial/tempo_multi)
        :.2f}x mais rapido")
76     print(f"Speedup Multi vs Vetorial    : {(tempo_vet/tempo_multi):.2f}
        x mais rapido")

```

3.2.2 Método de Leibniz

Para o método de Leibniz, realizou-se um *benchmark* variando a quantidade de termos (N) para observar o comportamento das diferentes abordagens sob cargas progressivas. Os resultados foram:

```

=====
SIMULACAO PARA N = 1000000
=====
Pi estimado (Multi)      : 3.14159165
Erro da aproximacao      : 0.00000100
Tempo Sequencial         : 0.3384 segundos

```

```

Tempo Vetorial      : 0.0266 segundos
Tempo Multi-Core    : 0.0295 segundos
Speedup Vetorial    : 12.72x mais rapido que serial
Speedup Multi-Core   : 11.46x mais rapido que serial
Multi vs Vetorial    : 0.90x de ganho/perda

```

```

=====
SIMULACAO PARA N = 5000000
=====

```

```

Pi estimado (Multi) : 3.14159245
Erro da aproximacao  : 0.00000020
Tempo Sequencial     : 1.7481 segundos
Tempo Vetorial        : 0.1512 segundos
Tempo Multi-Core      : 0.1155 segundos
Speedup Vetorial      : 11.56x mais rapido que serial
Speedup Multi-Core    : 15.13x mais rapido que serial
Multi vs Vetorial     : 1.31x de ganho/perda

```

```

=====
SIMULACAO PARA N = 10000000
=====

```

```

Pi estimado (Multi) : 3.14159255
Erro da aproximacao  : 0.00000010
Tempo Sequencial     : 3.7332 segundos
Tempo Vetorial        : 0.3239 segundos
Tempo Multi-Core      : 0.1693 segundos
Speedup Vetorial      : 11.52x mais rapido que serial
Speedup Multi-Core    : 22.06x mais rapido que serial
Multi vs Vetorial     : 1.91x de ganho/perda

```

Estes dados ilustram perfeitamente o impacto da carga computacional na viabilidade do paralelismo. Para $N = 1.000.000$, a abordagem puramente vetorial foi a mais eficiente, enquanto a multi-core sofreu uma perda de desempenho (0.90x) devido ao *overhead* do sistema operacional na criação dos processos. No entanto, ao elevar a carga para $N = 10.000.000$, o paralelismo se justificou amplamente: a versão multi-core superou a vetorial com um ganho de 1.91x e atingiu um notável *speedup* de 22.06x sobre a execução serial. Adicionalmente, observa-se que, com o aumento de termos, o erro absoluto da série decaiu em uma ordem de grandeza, atestando a convergência matemática do método.

O código-fonte utilizado para esta simulação é apresentado a seguir:

```

1 import time
2 import multiprocessing as mp
3 import numpy as np
4 import math
5
6 def worker_leibniz(intervalo):
7     inicio, fim = intervalo
8
9     # Usamos numpy internamente no worker para maxima velocidade
10    vetor_i = np.arange(inicio, fim)
11
12    # Calculo dos termos vetorizado para este bloco especifico
13    termos = ((-1.0)**vetor_i) / (2.0*vetor_i + 1.0)
14
15    return np.sum(termos)
16
17 if __name__ == '__main__':
18
19    valores_N = [10000000, 50000000, 100000000]
20
21    # Descubre quantos nucleos de CPU estao disponiveis
22    num_cores = mp.cpu_count()
23
24    print(f"Iniciando benchmark de Leibniz usando {num_cores} nucleos
25    ...\n")
26
27    for N in valores_N:
28        print("=====")
29        print(f"SIMULACAO PARA N = {N} ")
30        print("=====")
31
32        # SEQUENCIAL
33        inicio_tempo = time.time()
34        soma_seq = 0.0
35
36        for i in range(N):
37            termo = ((-1)**i) / (2*i + 1)
38            soma_seq += termo
39
40        pi_seq = 4 * soma_seq
41        tempo_seq = time.time() - inicio_tempo
42
43        # VETORIAL
44        inicio_tempo = time.time()
45
46        vetor_i = np.arange(N)
47        termos_vet = ((-1.0)**vetor_i) / (2.0*vetor_i + 1.0)

```

```

47     pi_vet = 4 * np.sum(termos_vet)
48
49     tempo_vet = time.time() - inicio_tempo
50
51     # MULTI-CORE
52     inicio_tempo = time.time()
53
54     # Divide o trabalho: cria os intervalos para cada nucleo
55     tamanho_bloco = N // num_cores
56     intervalos = []
57     for i in range(num_cores):
58         inicio_bloco = i * tamanho_bloco
59         # O ultimo nucleo pega o resto da divisao, se houver
60         fim_bloco = (i + 1) * tamanho_bloco if i != (num_cores - 1)
61         else N
62         intervalos.append((inicio_bloco, fim_bloco))
63
64     # Executa o mapeamento em paralelo
65     with mp.Pool(processes=num_cores) as pool:
66         resultados = pool.map(worker_leibniz, intervalos)
67
68     # Soma os resultados parciais de todos os nucleos
69     pi_multi = 4 * sum(resultados)
70
71     tempo_multi = time.time() - inicio_tempo
72
73     # ANALISE DE DESEMPENHO E ERRO
74     erro_absoluto = abs(math.pi - pi_vet)
75
76     print(f"Pi estimado (Multi)      : {pi_multi:.8f}")
77     print(f"Erro da aproximacao      : {erro_absoluto:.8f}")
78     print(f"Tempo Sequencial           : {tempo_seq:.4f} segundos")
79     print(f"Tempo Vetorial              : {tempo_vet:.4f} segundos")
80     print(f"Tempo Multi-Core             : {tempo_multi:.4f} segundos")
81
82     if tempo_vet > 0:
83         print(f"Speedup Vetorial          : {(tempo_seq/tempo_vet):.2f}
84               x mais rapido que serial")
85
86     if tempo_multi > 0:
87         print(f"Speedup Multi-Core         : {(tempo_seq/tempo_multi):.2f}
88               x mais rapido que serial")
89         print(f"Multi vs Vetorial          : {(tempo_vet/tempo_multi):.2f}
90               x de ganho/perda")
91     print("\n")

```

3.3 Comparação entre as implementações

A execução dos métodos em dois ambientes distintos (Octave e Python) revelou características marcantes de cada linguagem e de suas respectivas bibliotecas.

No Octave, a diferença de desempenho entre a abordagem serial e a vetorial foi extrema. Na simulação de Monte Carlo com apenas 150.000 pontos, a versão vetorial alcançou um *speedup* de mais de 466x. Isso ocorre porque o Octave é uma linguagem interpretada voltada especificamente para o cálculo matricial. Consequentemente, laços *for* puros são processados de forma muito ineficiente pelo interpretador, enquanto operações com vetores disparam instruções SIMD diretamente no hardware. A desvantagem observada no Octave foi a limitação do ambiente em nuvem, que inviabilizou o teste de multiprocessamento.

Por outro lado, o Python mostrou-se um ambiente mais flexível. O seu laço serial padrão é comparativamente mais rápido que o do Octave para lógicas puras, o que fez com que o *speedup* da vetorização com NumPy fosse menor em termos relativos (cerca de 6x para Monte Carlo), embora o tempo absoluto tenha sido excelente (processando 10 milhões de pontos em apenas 0.39 segundos). A grande vantagem do Python nesta prática foi a facilidade de implementar paralelismo utilizando a biblioteca *multiprocessing*, permitindo a exploração de todos os 8 núcleos do processador e o teste dos limites do *overhead* do sistema operacional.

4 Discussão dos Resultados

Os resultados experimentais confirmam os conceitos teóricos, especialmente no que tange à vetorização, ao paralelismo e à Lei de Amdahl.

A **vetorização (SIMD)** provou ser a técnica de otimização de melhor custo-benefício. Em todos os cenários (Octave e Python, Monte Carlo e Leibniz), a transição de um algoritmo serial para um algoritmo vetorial reduziu o tempo de execução em ordens de grandeza, sem a necessidade de gerenciar múltiplos processos. Isso demonstra o poder das Unidades Lógicas e Aritméticas (ALUs) modernas em aplicar uma mesma instrução a múltiplos dados em um único ciclo de *clock*.

Um ponto crucial na interpretação dos resultados é a aparente violação da Lei de Amdahl. Ao observar acelerações (*speedups*) de 466x no Octave ou 22x no Python com apenas 8 núcleos físicos, pode-se erroneamente concluir que o ganho superou o limite teórico estabelecido pelo número de processadores. Contudo, esses resultados não violam a lei matemática; eles são o reflexo da sobreposição de diferentes níveis de otimização e da eliminação de gargalos de *software*.

O ganho extraordinário medido não advém apenas da distribuição de tarefas, mas de três fatores combinados:

1. **Eliminação do *Overhead* do Interpretador:** Tanto o Python quanto o Octave

são linguagens interpretadas. Em uma abordagem serial, o interpretador realiza checagens de tipos e alocações dinâmicas a cada iteração do laço `for`. Ao vetorizar o código (usando bibliotecas como o NumPy), essas operações são delegadas a rotinas pré-compiladas em C ou Fortran de altíssimo desempenho, eliminando o gargalo da interpretação.

2. **Paralelismo em nível de instrução (SIMD):** O ganho absurdo no Octave e o ganho base no Python ocorrem majoritariamente dentro de um único núcleo de processamento. Utilizando registradores largos (como os da família AVX), o processador consegue empacotar e calcular diversos valores de ponto flutuante simultaneamente, gerando um multiplicador de desempenho que independe da quantidade de núcleos MIMD da máquina.
3. **Efeito Cascata (Multiplicativo):** O *speedup* global é o produto das várias camadas de otimização. Na simulação de Leibniz com $N = 10.000.000$ em Python, a transição do laço serial para operações vetorizadas proporcionou um ganho base de 11.52x. Quando esse código vetorizado foi paralelizado em 8 núcleos físicos, o ganho adicional isolado foi de apenas 1.91x (estritamente menor que os 8 processadores disponíveis, o que respeita rigorosamente a restrição da Lei de Amdahl para a fração paralelizável e seus *overheads* de comunicação). O *speedup* reportado de 22x é resultado do produto dessas otimizações ($11.52 \times 1.91 \approx 22.00$).

O **multiprocessamento**, avaliado no Python, trouxe à tona a realidade do custo de paralelização (*overhead*). Ao observar a simulação de Monte Carlo, a utilização de 8 núcleos resultou em um ganho de apenas 1.26x sobre a versão puramente vetorial. O tempo gasto pelo sistema operacional para instanciar 8 processos distintos, copiar os dados para as memórias locais e reduzir os resultados em uma única soma competiu fortemente com o tempo de cálculo da simulação.

Esse fenômeno ficou ainda mais evidente no método de Leibniz. Para uma carga de $N = 1.000.000$, a versão multi-core foi mais lenta que a vetorial (perda de 0.90x), caracterizando um cenário onde o paralelismo é injustificável. Somente ao escalar o problema para $N = 10.000.000$, o volume de cálculos superou as penalidades de comunicação, viabilizando a abordagem multi-core.

Do ponto de vista numérico, a análise de erro validou a natureza dos métodos: Monte Carlo apresentou pequenas flutuações estatísticas dependentes da geração de números pseudoaleatórios, com baixa eficiência computacional para ganhar novas casas decimais. Já a série de Leibniz apresentou um decaimento determinístico do erro, porém com uma taxa de convergência notoriamente lenta, demandando dezenas de milhões de iterações para garantir precisão além da sexta casa decimal de π .

5 Conclusões

A presente prática permitiu compreender, desenvolver e avaliar o impacto das estratégias de programação na exploração da arquitetura dos processadores modernos. A simples tradução de uma fórmula matemática para o código na forma de laços de repetição (abordagem serial) subutiliza drasticamente o potencial do *hardware*.

Conclui-se que, para problemas matemáticos e de simulação com grandes volumes de dados independentes, a **vetorização** deve ser a primeira técnica de otimização a ser adotada, dada a sua simplicidade de implementação (através de bibliotecas como NumPy em Python ou sintaxe nativa no Octave) e o seu gigantesco retorno em desempenho.

O **paralelismo em múltiplos núcleos**, por sua vez, é uma ferramenta poderosa, mas exige projeto cuidadoso. Como demonstrado pelos testes de *overhead*, dividir uma tarefa entre vários núcleos de processamento só gera aceleração (*speedup*) se a carga computacional do problema for grande o suficiente para diluir o custo de criação e sincronização dos processos imposto pelo sistema operacional.

Finalmente, a prática atestou a aplicabilidade dos métodos de Monte Carlo e da Série de Leibniz, ressaltando que a escolha do algoritmo deve ser guiada não apenas pela sua formulação teórica, mas pela forma como ele mapeia as instruções na memória, nos registradores vetoriais e nos múltiplos núcleos de processamento disponíveis na máquina hospedeira.

6 Referências

1. HENNESSY, J. L.; PATTERSON, D. A. **Arquitetura de Computadores: Uma Abordagem Quantitativa**. 3. ed. Rio de Janeiro: Campus, 2003.
2. PEDRINO, E. C. **Conceitos de Arquiteturas Avançadas - Aula 1**. UFSCar, 2026.
3. PRÁTICA 1. **Cálculo de Pi por Monte Carlo**. Roteiro da disciplina Arquiteturas de Alto Desempenho, UFSCar, 2026.
4. PRÁTICA 2. **Cálculo de Pi por Leibniz**. Roteiro da disciplina Arquiteturas de Alto Desempenho, UFSCar, 2026.
5. Python Software Foundation. **Multiprocessing**. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em: 2026.
6. Material de apoio do Octave sobre manipulação vetorial.